Implementation of APB-UART Design

Introduction:

The APB-UART design demonstrates a simple and clear communication path between an Advanced Peripheral Bus (APB) interface and a UART-style peripheral. The purpose of this design is to show how a processor or controller can use the APB protocol to write data into a UART transmit register and read the same data back from a UART receive register using memory-mapped register access. The design operates using a 50 MHz system clock and models UART-side transfer internally through register movement. In this implementation, data written through the APB interface is passed into the UART block, stored internally, and made available at the receive side for APB readback. The complete design consists of four modules: uart, apb_uart_bridge, apb_uart_top, and apb_uart_tb.

Design Code Description:

The APB-UART design is organized as a simple hierarchical system with apb_uart_top as the top-level integration module. This top module instantiates apb_uart_bridge, which acts as the protocol interface between the APB bus and the UART register block. The APB bridge decodes APB read and write transactions, maps them to UART register operations, and returns the corresponding data to the APB master. The uart module used in this design is a simplified register-based UART model. Instead of implementing complete serial transmission and reception logic, it demonstrates UART behavior by storing transmitted data in an internal transmit register and forwarding it directly to a receive register for observation. This allows the design to clearly demonstrate APB write, APB read, register access, and end-to-end data flow without introducing unnecessary serial timing complexity. The APB interface provides memory-mapped access to UART registers using standard APB control and data signals such as PSEL, PENABLE, PWRITE, PADDR, PWDATA, PRDATA, and PREADY.

APB-UART Block Diagram

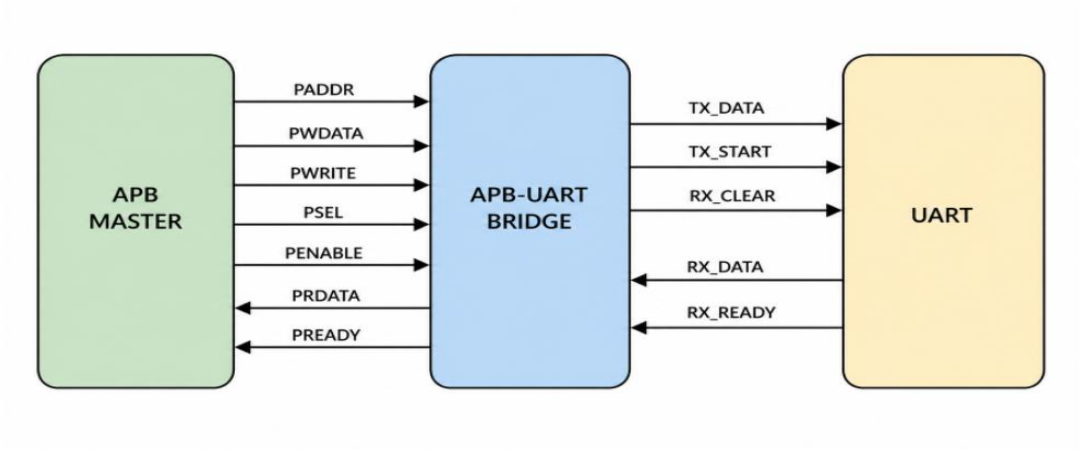

Fig: APB-UART Bridge

System Architecture

1. UART Module (uart.v)

The uart module is a simplified UART-style peripheral used to demonstrate data movement between transmit and receive paths. It accepts transmit data (tx_data) and a write enable (tx_write) from the APB bridge. When valid transmit data is written, the UART captures the byte into an internal transmit register and immediately forwards it to the receive output register. This models UART communication in a simplified way for educational purposes. The module also generates two status signals: rx_ready, which indicates valid received data is available, and tx_busy, which indicates transmit activity.

2. APB-UART Bridge (apb_uart_bridge.v)

The apb_uart_bridge module acts as the communication bridge between the APB bus and the UART block. It receives APB transactions from the bus master and converts them into UART register operations. During a write transaction, the bridge decodes the APB address and writes the lower 8 bits of PWDATA into the UART transmit register. During a read transaction, it returns either received UART data or UART status depending on the selected address. The bridge handles APB protocol sequencing using PSEL, PENABLE, and PREADY, ensuring proper APB setup and access phases. This module is the key element that enables APB-controlled UART communication.

3. Top-Level Integration (apb_uart_top.v)

The apb_uart_top module is the top-level wrapper of the design. It connects the APB interface signals from the testbench or processor side to the apb_uart_bridge module. This module serves as the integration point for the complete APB-UART system and exposes a clean APB interface externally. Internally, it simply instantiates the APB bridge and connects all required signals.

4. Testbench (apb_uart_tb.v)

The apb_uart_tb module verifies the functionality of the APB-UART design. It generates the system clock, applies reset, performs APB write and read transactions, and prints results to the terminal. The testbench writes data into the UART transmit register through APB, then reads the UART receive register and status register to confirm proper operation. Two data transfers are performed (0xA5 and 0x3C) to demonstrate repeated APB-to-UART communication. The testbench also generates waveform output in apb_uart_dump.vcd for visual verification using GTKWave.

Register Map

Address	Access	Description
0x00	Write	TX Data Register – Write data to UART
0x04	Read	RX Data Register – Read received UART data
0x08	Read	Status Register – {tx_busy, rx_ready}

Timing Specifications:

The design operates with a 50 MHz system clock, corresponding to a clock period of 20 ns. Since this implementation is focused on APB-to-UART register communication rather than full serial transmission, no baud-rate generator is required. UART transfer is modeled internally as register movement and therefore completes in synchronous clock cycles. This keeps the design simple and focused on APB transaction behavior.

APB-UART Communication Flow

The communication begins when the APB master initiates a write transaction to the UART transmit register at address 0x00. During the APB setup phase, the master places the target address on PADDR, transmit data on PWDATA, asserts PWRITE, and selects the peripheral using PSEL. In the next cycle, the APB access phase begins when PENABLE is asserted. The APB bridge detects this valid write transaction, asserts PREADY, and forwards the lower 8 bits of PWDATA to the UART as tx_data while pulsing tx_write.

Inside the UART block, the incoming byte is stored into the internal transmit register and immediately copied to the receive register. At the same time, the UART raises rx_ready to indicate valid receive data is available. This models successful UART transmission and reception in a simplified form.

Next, the APB master performs a read transaction at address 0x04 to fetch the UART receive data. The bridge decodes the read request and places the received UART byte on PRDATA, which is returned to the APB master. A second read at address 0x08 returns the UART status register, containing tx_busy and rx_ready. This demonstrates how APB is used to control UART communication through memory-mapped register access.

Procedure To Create a Lab:

Step 1: Set up Project Directory

1.1: Create directory for project: `mkdir apb_uart_interface`

1.2: Open project directory: `cd apb_uart_interface`

Step 2: Create UART Design file

2.1: Open gvim editor for UART design with command: `gvim uart.v`

UART Design Code:

```
module uart (
    input  wire       clk,
    input  wire       rst_n,

    input  wire [7:0] tx_data,
    input  wire       tx_write,

    output reg  [7:0] rx_data,
    output reg        rx_ready,
    output reg        tx_busy
);

reg [7:0] tx_reg;

always @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        tx_reg   <= 8'h00;
        rx_data  <= 8'h00;
        rx_ready <= 1'b0;
        tx_busy  <= 1'b0;
    end else begin
        rx_ready <= 1'b0;

        if (tx_write) begin
            tx_reg   <= tx_data;
            tx_busy  <= 1'b1;

            rx_data  <= tx_data;
            rx_ready <= 1'b1;

            tx_busy  <= 1'b0;
        end
    end
end

endmodule
```

2.2: Save the file using command- `:wq`

Step 3: Create APB UART Bridge Design file

3.1: Open gvim editor for APB UART bridge with command:

```
gvim apb_uart_bridge.v
```

APB Bridge Design Code:

```
module apb_uart_bridge (
    input  wire        clk,
    input  wire        rst_n,

    input  wire [31:0] PADDR,
    input  wire [31:0] PWDATA,
    input  wire        PWRITE,
    input  wire        PSEL,
    input  wire        PENABLE,

    output reg  [31:0] PRDATA,
    output reg         PREADY
);

reg  [7:0] tx_data;
reg        tx_write;

wire [7:0] rx_data;
wire       rx_ready;
wire       tx_busy;

uart uart_inst (
    .clk(clk),
    .rst_n(rst_n),
    .tx_data(tx_data),
    .tx_write(tx_write),
    .rx_data(rx_data),
    .rx_ready(rx_ready),
    .tx_busy(tx_busy)
);

always @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        PREADY   <= 1'b0;
        PRDATA   <= 32'h0;
        tx_data  <= 8'h00;
        tx_write <= 1'b0;
    end else begin
```

```
        PREADY   <= 1'b0;
        tx_write <= 1'b0;

        if (PSEL && PENABLE) begin
            PREADY <= 1'b1;

            if (PWRITE) begin
                case (PADDR[7:0])
                    8'h00: begin
                        tx_data  <= PWDATA[7:0];
                        tx_write <= 1'b1;
                    end
                endcase
            end else begin
                case (PADDR[7:0])
                  8'h04: PRDATA <= {24'h0, rx_data};// RX data
                  8'h08: PRDATA <= {30'h0, tx_busy, rx_ready};
                  default: PRDATA <= 32'h0;
               endcase
            end
        end
    end
end

endmodule
```

3.2: Save the file using command- `:wq!`

Step 4: Create Top-Level Design File

4.1: Open gvim editor for top-level design with command: `gvim apb_uart_top.v`

Top-Level Design Code:

```
module apb_uart_top (
    input  wire        clk,
    input  wire        rst_n,
    input  wire [31:0] PADDR,
    input  wire [31:0] PWDATA,
    input  wire        PWRITE,
    input  wire        PSEL,
    input  wire        PENABLE,

    output wire [31:0] PRDATA,
    output wire        PREADY
);
```

```
apb_uart_bridge dut (
    .clk(clk),
    .rst_n(rst_n),
    .PADDR(PADDR),
    .PWDATA(PWDATA),
    .PWRITE(PWRITE),
    .PSEL(PSEL),
    .PENABLE(PENABLE),
    .PRDATA(PRDATA),
    .PREADY(PREADY)
);
Endmodule
```

4.2: Save the file using command-: `wq!`

Step 5: Create Testbench file

5.1: Open gvim editor for testbench with command: `gvim apb_uart_tb.v`

Testbench Code:

```
`timescale 1ns/1ps

module apb_uart_tb;
reg clk = 0;
reg rst_n;
reg [31:0] PADDR;
reg [31:0] PWDATA;
reg        PWRITE;
reg        PSEL;
reg        PENABLE;
wire [31:0] PRDATA;
wire        PREADY;

apb_uart_top dut (
    .clk(clk),
    .rst_n(rst_n),
    .PADDR(PADDR),
    .PWDATA(PWDATA),
    .PWRITE(PWRITE),
    .PSEL(PSEL),
    .PENABLE(PENABLE),
    .PRDATA(PRDATA),
    .PREADY(PREADY)
);
```

```
always #10 clk = ~clk;
initial begin
    rst_n   = 0;
    PADDR   = 0;
    PWDATA  = 0;
    PWRITE  = 0;
    PSEL    = 0;
    PENABLE = 0;

    #100;
    rst_n = 1;

    // Write TX register
    apb_write(32'h00, 32'hA5);

    // Read RX register
    apb_read(32'h04);

    // Read STATUS register
    apb_read(32'h08);

    // Second transfer
    apb_write(32'h00, 32'h3C);
    apb_read(32'h04);
    apb_read(32'h08);

    #100;
    $finish;
end

task apb_write(input [31:0] addr, input [31:0] data);
begin
    @(posedge clk);
    PADDR   = addr;
    PWDATA  = data;
    PWRITE  = 1;
    PSEL    = 1;
    PENABLE = 0;

    @(posedge clk);
    PENABLE = 1;
    @(posedge clk);
    PSEL    = 0;
```

```
    PENABLE = 0;
    $display("APB WRITE : ADDR=%h DATA=%h", addr, data);
end
endtask

task apb_read(input [31:0] addr);
begin
    @(posedge clk);
    PADDR   = addr;
    PWRITE  = 0;
    PSEL    = 1;
    PENABLE = 0;

    @(posedge clk);
    PENABLE = 1;

    @(posedge clk);
    @(posedge clk);

    $display("APB READ  : ADDR=%h DATA=%h", addr, PRDATA);

    PSEL    = 0;
    PENABLE = 0;
end
endtask

initial begin
    $dumpfile("apb_uart_dump.vcd");
    $dumpvars();
end

endmodule
```

5.2: Save the testbench file using command- `:wq!`

5.3: Check the saved files in current directory: use command `ls`

Step 6: Simulation Script

6.1: Create a simulation script: `gvim run.sh`

```
#!/bin/bash

# Step 1: Compile RTL and Testbench using Verilator

verilator --binary -j 0 -Wall uart.v apb_uart_bridge.v
apb_uart_top.v apb_uart_tb.v --top apb_uart_tb --timing --
trace --CFLAGS "-std=c++20"

# Step 2: Enter build directory

cd obj_dir || { echo "Error: obj_dir not found"; exit 1; }

# Step 3: Build simulation executable

make -f Vapb_uart_tb.mk Vapb_uart_tb || { echo "Error:
Compilation failed"; exit 1; }

# Step 4: Run simulation

./Vapb_uart_tb || { echo "Error: Simulation failed"; exit 1; }

# Step 5: Open waveform

gtkwave apb_uart_dump.vcd
```

6.2 : Make the script file executable : `chmod +x run.sh`

6.3 : Run the simulation: `./run.sh`

Simulation Output:

The terminal output confirms successful APB-to-UART communication. The APB master writes 0xA5 to the UART transmit register and reads back 0xA5 from the UART receive register. It then repeats the same sequence with 0x3C. This verifies correct APB write decoding, UART data transfer, and APB readback.

```
APB WRITE : ADDR=00000000 DATA=000000a5
APB READ  : ADDR=00000004 DATA=000000a5
APB READ  : ADDR=00000008 DATA=00000000
APB WRITE : ADDR=00000000 DATA=0000003c
APB READ  : ADDR=00000004 DATA=0000003c
APB READ  : ADDR=00000008 DATA=00000000
```

Output Waveform:

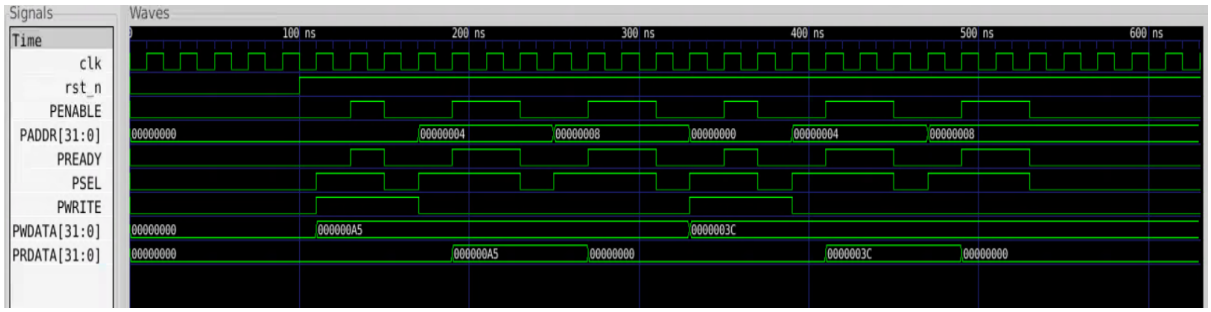

Fig: Waveform Output

The waveform confirms correct APB transaction sequencing and UART register behavior. During each APB write, PSEL is asserted first to indicate the setup phase, followed by PENABLE in the next cycle to indicate the access phase. At this point, the APB bridge asserts PREADY, completing the transaction. The write data placed on PWDATA is captured and transferred into the UART transmit path.

During APB reads, the same APB setup and access sequence is observed. When the read address is 0x04, the bridge returns the UART receive data on PRDATA. The waveform shows that the data written during the earlier APB write is correctly returned during the APB read. A subsequent read at 0x08 returns the UART status register. The waveform clearly demonstrates correct APB protocol timing, proper register decoding, and successful end-to-end data flow from APB write to UART storage and APB readback.

Applications

This APB-UART design is useful for understanding how APB peripherals are controlled in SoC and embedded systems. It demonstrates how processors communicate with peripherals using memory-mapped registers. The same concept is widely used in practical SoC designs for UARTs, timers, GPIO, SPI, and I2C peripherals. This simplified APB-UART example provides a clear foundation for learning APB-based peripheral integration before moving to full UART serial protocol implementations.

Conclusion

The APB-UART design successfully demonstrates how APB protocol transactions can be used to communicate with a UART-style peripheral through memory-mapped registers. The design clearly shows the relationship between APB write/read operations and UART data movement, making it easy to understand how peripheral communication is implemented in SoC designs. The simulation and waveform results confirm correct APB protocol handling, accurate register access, and successful data transfer across the complete design.